DELAUNAY TRIANGULATION FOR GREEDY k-CENTRE FACILITY PLACEMENT

MINOR PROJECT REPORT

Submitted in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

Department of Computer Science and Engineering

Submitted to

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)



Submitted by

Jayesh Kumar (24U020018 V Semester)
Atharv Agrawal (24U020121 V Semester)

Under the supervision of

Dr. Neeta Anna Eapen
Assistant Professor
(Computer Science and Engineering)

October 2026

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)



CERTIFICATE

This is to certify that the work embodied in this report entitled **"Delaunay Triangulation for Greedy k-Centre Facility Placement"** has been satisfactorily completed by Atharv Agrawal (24U020121) and Jayesh Kumar (24U020018). It is a bonafide piece of work, carried out under our guidance in the **Department of Computer Science and Engineering**, **Indian Institute of Information Technology, Bhopal** for the partial fulfillment of the Bachelor of Technology during the academic year 2026-27.

Dr. Neeta Anna Eapen,
Assistant Professor,
Department of CSE,
IIIT Bhopal (M.P.)

Dr. Rahul Chaurasia,
Department of CSE,
IIIT Bhopal (M.P.)



INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)

DECLARATION

We hereby declare that the following minor project report entitled "Delaunay Triangulation for Greedy k-Centre Facility Placement" presented here is in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering**. It is an authentic documentation of our original work carried out under the able guidance of **Dr. Neeta Anna Eapen**. The work has been carried out entirely at the Indian Institute of Information Technology, Bhopal. The project work presented has not been submitted in part or whole to award of any degree or professional diploma in any other institute or organization.

We, with this, declare that the facts mentioned above are true to the best of our knowledge. In case of any unlikely discrepancy that may occur, we will be the ones to take responsibility.

Atharv Agrawal (24U020121)
Jayesh Kumar (24U020018)

ACKNOWLEDGEMENT

We, Atharv Agrawal and Jayesh Kumar, would like to thank everyone who helped and directed us in successful completion of our minor project "Delaunay Triangulation for Greedy k-Centre Facility Placement".

First of all we would like to thank our esteemed Director, Prof. Dr. Ashutosh Kumar Singh for giving us the opportunity to undertake our project by giving us the resource and facilities for doing the project. His vision and enthusiasm has kept us motivated through out the completion of the project.

We are highly thankful to our Project Guide, Dr. Neeta Anna Eapen for her round the clock guidance, suggestions, and valuable advice through out the project. We, grateful to her knowledge and inputs, which enabled us to make our project more effective and we are motivated by her support.

We would like to thank our Department Coordinator Dr. Rahul Chaurasia, for the support and encour- agement given while working on this project.

Besides, we would like to thank all the faculties of the CSE for their precious guidance and support in clarifying our doubts and providing the right direction.

Thanks to our friends for their constant support and suggestions, and helping us through many stages of this project. Their suggestions and support kept us engaged and motivated throughout the project work.

And finally, and not the least we would like to thank our parents and families for their love, patience and encouragement and for the moral support throughout the project. Their understanding towards us and support were the prime motivator to what we are today.

We thank you all dearly for all the help that you gave us directly and indirectly to make this project a success. Without your help and support this project would not have taken place.

Atharv Agrawal[24U020121]

Jayesh Kumar[24U020018]

AREA OF WORK

This project focuses on computational geometry and approximation algorithms, using a set of tools and techniques required for building fast and verifiable spatial algorithms.

One may argue that every supporting computation in a greedy k-centre pipeline could be done with straightforward scans over all points, and that is exactly the catch. Choosing a good first centre, detecting outliers and locating the farthest uncovered spot each have a simple implementation that costs O(n2) or relies on an approximate grid. A single Delaunay triangulation built once in O(nlogn), replaces all three with faster geometric operations.

This project is about how the dual Delaunay triangulation and Voronoi diagram can be placed over Gonzalezs greedy farthest point algorithm for facility placement and it Tests the impact of this layer on speed, solution quality, and outliers, using Python and SciPy.

TABLE OF CONTENT

S.no	Title	Page No.
	Certificate	
	Declaration	
	Abstract	
1	Introduction	1
2	Literature Review or Survey	2
3	Problem Definition And Objective	3
4	Proposed Methodology And Work	4
5	Proposed Algorithm	6
6	Proposed flowchart/ DFD/ Block Diagram	8
7	Implementation (Tools & Technology Used)	11
8	Result Analysis	13
9	Conclusion & Future Scope	17
10	References	19

LIST OF FIGURES

Fig. No.	Description	Page No.
4.1	System architecture of the Delaunay-augmented k-centre pipeline	4
6.1	Training / execution flowchart of the proposed pipeline	8
6.2	Component block diagram	8
6.3	Data flow diagram (Level 0)	10
8.1	First-point strategy: mean radius over instances	13
8.2	Plain Gonzalez versus Delaunay filter + Gonzalez	14
8.3	Cost of covering the last few points, and detector precision/recall/F1	14
8.4	Next centre via Delaunay circumcentres (largest empty circle)	15
8.5	Measured scaling of runtime against n (log-log)	16

LIST OF TABLES

Table No.	Description	Page No.
4.1	Experimental configuration	4
5.1	Time complexity of all components in the project	7
7.1	Code structure	11
8.1	Seeding-rule findings and evidence	13
8.2	Outlier detector versus MAD threshold	14
8.3	Outlier-handling methods compared (n = 250, z = 8)	15
8.4	Grid scan versus Delaunay circumcentres	15
8.5	Stated complexity versus fitted exponent	16

ABSTRACT

In the case of the k-centre problem, this is equivalent to choosing k locations such that the maximum distance from each demand point to the nearest centre is minimised, which farthest-greedy Gonzalez's algorithm solves in O(nk) time with at most twice the optimal radius. For the case of the k-centre problem, however, the constant time complexity guarantee reveals nothing about what point should be used as the first centre, what we do with stray outliers, or how we find the farthest uncovered point on a continuous map.

This work introduces a Delaunay implementation of a geometric component to the greedy baseline. It can be exploited by one triangulation constructed in time O(n log n) for three tasks: to seed the first centre as a minimum spanning tree (the Euclidean minimum spanning tree is included in the Delaunay triangulation), to compute outliers scores using the nearestneighbour distance (which can be obtained from any mesh edge), and to find the biggest empty circle based on Delaunay circumcentres (the Voronoi vertices).

Experimental results on five synthetic dataset families (n = 400, k = 6, 12 instances per family) show. Which is not to say that a cheap deterministic rule for seeding always beats a random start - but computing Via Delaunay on the MST-centre seed takes only 10× as long (and gives the same answer).

Lower outlier score at minimum incident-edge lifts detection F1 to 0.89. Circumcentre search produces 619× fewer candidates, runs 55× faster than 500 × 500 grid scan 99.98% agreement. Empirical growth exponents of 1.02 vs 2.25 for Delaunay MST seed vs dense MST seed establish O(n 2) to O(n log n) improvement.

Keywords: k-Centre, Facility Placement, Gonzalez Algorithm, Delaunay Triangulation, Voronoi Diagram, Minimum Spanning Tree, Outliers, Largest Empty Circle, Computational Geometry.

CHAPTER 1. INTRODUCTION

1.1 Background

Location Facility location is the process of selecting the locations for a pre-specified number of facilities to be located, to provide a defined level of service to a set of demand points. The k -centre problem is to find the k centres that minimize the covering radius.

1.2 Gaps Around the Greedy Baseline

The 2× guarantee does not depend where we start, but the actual radius obtained can vary greatly depending on it. Greedy methods also fling centres towards outlying singleton points and their supporting calculations are costly if implemented naively: an MST-centre seed is O(n2) for an n×n dense distance matrix outlier detection requires pairwise distances or an alternate index and the farthest uncovered point on a continuous map is in practice approximated by an O(res2k) grid scan.

1.3 Delaunay Triangulation as a Unifying Layer

The three properties the Delaunay triangulation [4][5] are that it can be constructed in O(n log n) and, it has the following three properties:
As a subgraph of Euclidean minimum spanning tree that is Kruskal's algorithm [9] look at is. Only on the order of 3n edges. The nearest-neighbour graph is also contained in it, so every point's.
The shortest incident edge of nearest-neighbour distance is it. The vertices of the dual Voronoi In the diagram are the circumcentres of the Delaunay triangles so provide a small, precise candidate set for the largest empty circle.

1.4 Project Motivation and Scope

Basic gonzo guarantees three costly replacements while only being a proof of concept.
Supporting scans and performing operations on a common triangulation, and to assess the outcome by Machine learning de maker. Experiments and fitted runtimes. The scope is the Euclidean plane with synthetic data;
Fair, streaming and capacitated variants are omitted for now.

CHAPTER 2. LITERATURE REVIEW

2.1 Approximation Algorithms for k-Centre

Gonzalez[1] proposed the farthest-point traversal a 2-approximations in O(nk) time Hochbaum and Shmoys[2] obtain the same factor by parametric pruning in O(n2logn). As neither factors can be improved most practical improvement can be achieved by seeding local refinement and faster supporting calculation.Lloyd's kmeans[11] with the k-means++ seeding[10] is a different objective used only as a benchmark.

2.2 k-Centre with Outliers

Since k-centre minimises a maximum, a few far away points could dominate the solution.
Charikar et al[3] provided a greedy 3-approximation which has gaps of up to z uncovered points, at O(nLog n) cost, with z the number of sites known a priori. Cheap heuristics that avoid the z furthest sites exist.
No guarantee. Median of absolute deviation (MAD) can be used to set robust threshold.

2.3 Delaunay Triangulation and Voronoi Diagrams

The Delaunay triangulation [4] is the dual of the Voronoi diagram, and standard texts [5] cover its properties, including containment of the Euclidean MST and the nearest-neighbour graph. Quickhull [6], exposed through SciPy [13], builds it in O(n log n), while the incremental Bowyer–Watson algorithm [7][8] is simple but $O(n^2)$ with naive point location. Both are used in this project.

2.4 Summary and Research Gap

Previously work provides robust guarantees for k-centre but considers seeding outlier detection and farthest-point search independently. There is limited experimental evidence about how one. Whether shared Delaunay triangulation will speed up those steps or whether geometric seeding can outperform a random start. This project provides that evidence.

CHAPTER 3. PROBLEM DEFINITION AND OBJECTIVES

3.1 Problem Statement

The goal of the project is to optimize several main procedures of the greedy **k-centre** algorithm, such as initial center selection, farthest-point search, and methods of outliers processing.
The project further tests the Gonzalez algorithm's known 2-approximation for the metric **k-centre** problem, as a function of initial centroid choice, outliers, etc. and compare the time-cost of dense vs grid search as well as empirically confirm time-complexity estimates.

3.2 Formal Problem Formulation

Let X = {x1,..., xn} be n points in the plane and k a positive integer. The k-centre problem asks to find C X with |C| = k, that minimise r(C) = max over x X of min over c C of d(x, c) where d is the Euclidean distance.In the outlier variant up to z points may be left uncovered and r(C) is taken over the remaining n − z points. In the continuous variant a centre may be placed anywhere in the plane.

3.3 Project Objectives

- Implement Gonzalez's greedy algorithm and its variants, plus Hochbaum–Shmoys, brute force and Charikar's outlier greedy for comparison.
- Build a Delaunay layer for MST-centre seeding, 1-NN outlier scoring and largest-empty-circle search.
- Compare seven seeding rules on five dataset families and tune a Delaunay outlier detector by precision, recall and F1.
- Fit measured runtimes to verify the stated complexity of the main routines.

3.4 Scope and Constraints

All experiments use synthetic two-dimensional data with Euclidean distance. The browser simulator uses an $O(n^2)$ Bowyer–Watson triangulation, while the Python implementation uses Qhull in O(n logn). Capacitated, fair, streaming and road-network variants are outside the scope.

CHAPTER 4. PROPOSED METHODOLOGY AND WORK DESCRIPTION

4.1 Overall Architecture

The main algorithm remains Gonzalez's greedy farthest-point method (O(nk), 2× guarantee). The new part is a Delaunay layer built once in O(n log n) that supports three tasks, each of which would otherwise need a separate expensive scan (Figure 4.1).

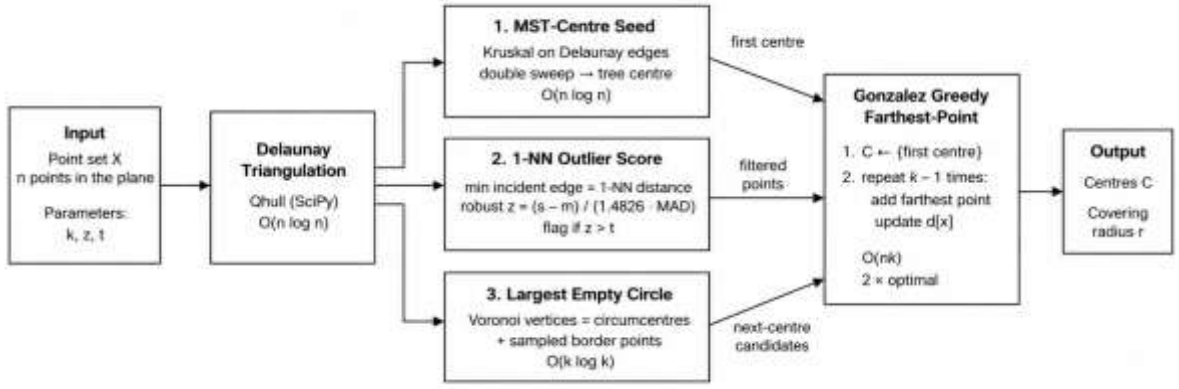


Figure 4.1: Architecture of the Delaunay-augmented k-centre pipeline.

4.2 Datasets and Experimental Configuration

All data is synthetic, drawn from five families: uniform, clustered, clustered with outliers, two-moons (non-convex) and field-with-hole.

Table 4.1: Experimental configuration

Parameter	Value
Points / centres / instances	n = 400, k = 6, 12 instances per family
Seeding rules compared	7 single-start rules (random, centroid-nearest, farthest-from-centroid, KD-tree density, Delaunay density, Delaunay MST centre, largest empty circle) plus 25-start multi-start as reference
Outlier experiment	k = 6, 10 planted outliers, MAD threshold sweep
Scaling experiment	n = 100 … 3200, 6 runs, exponent fitted for n ≥ 400

4.3 Delaunay-Accelerated First-Centre Seeding

The MST centre is found by building the Euclidean MST and locating its middle with a double sweep. A dense implementation is $O(n^2)$. Since the Euclidean MST is a subgraph of the Delaunay triangulation, Kruskal's algorithm can run on the roughly 3n Delaunay edges in O(n log n), giving the same tree and therefore the same chosen point.

4.4 Delaunay-Based Outlier Detection

A stray point has no close neighbour. The first detector used the mean incident-edge length, but border points of clusters have long edges across gaps and were wrongly flagged (F1 = 0.63). Using the minimum incident edge length, which equals the 1-NN distance because the Delaunay graph contains the nearest-neighbour graph, is read from the mesh in O(n). Scores are standardised with a MAD-based robust sigma and thresholded. Flagged points are removed, Gonzalez runs on the rest and the radius is scored on all points. Unlike robust Gonzalez or Charikar, the filter needs no z.

4.5 Largest Empty Circle via Circumcentres

Each Gonzalez step needs the point farthest from all current centres. The Voronoi vertices of the centres are the circumcentres of their Delaunay triangles, so only these O(k) vertices plus a sampled border scan need be scored, instead of a 500 × 500 grid.

4.6 Evaluation Protocol

Seeding is evaluated by mean maximum radius for 12 seeds per family. Outlier detection by precision recall and F1 Against planted truth and by radius as z grows. Speed is evaluated by wall-clock time and candidates scored. Complexity is determined by the fit of a power law to six runs per routine for n 400 where the python overhead is not as large.

CHAPTER 5. PROPOSED ALGORITHMS

5.1 Gonzalez Greedy Farthest-Point (Baseline)

Algorithm 1: Gonzalez(X,k, first)

1. C {first} take d[x] as the distance from x to it nearest centre.
2. Repeat k 1 times choose x equals argmax d[x],C C{x},d[x]min(d[x],dist(x, x*)).
3. Return C and r=max d[x].

Every iteration of the k-iteration is a linear scan, hence total cost n k, solution at most twice optimal for any first centre.

5.2 MST-Centre Seeding by Delaunay

Algorithm 2: DelaunayMSTSeed(X)

1. Construct the Delaunay triangulation of X with Qhull, O(n log n).
2. Sort its 3n edges, run Kruskal [9] union-find to find the Euclidean MST.
3. Double sweep: pick a vertex, find the vertex furthest away, find the vertex furthest away, the mid of the two is the MST centre.

Since the Euclidean MST is nested within the triangulation, the tree and the selected point are actually the same as the dense case, at O(n log n) instead of $O(n^2)$.

5.3 Delaunay Outlier Filter + Gonzalez

Algorithm 3 Delaunay Filter Gonzalez(X,k, t)

1. Triangulate; s(p) = length of shortest incident edge of p (1-NN), O(n).
2. Where to go median m and median-absolute-deviation MAD of the scores. Robust z(p) = (s(p) m) / (1.4826 MAD).
3. If (flag p as outlier) if (z(p) > t) (t is 8.0 in our experiments) Run Gonzalez on all unflagged points.
4. Report the radius over all n points, so removed points still scored.

5.4 Largest Empty Circle Using Circumcentres

Algorithm 4: Next Centre(X, C)

1. Triangulate the centres C, O(k log k), and locate the (interior) circumcentre of each triangle.
2. Add sample points along the edges of each Voronoi diagram that cross the border of the map.
3. Return the candidate that was dropped at the furthest distance to its closest centre.

About 404 candidates are scored instead of 250,000 grid cells. Sampled border scan can be replaced with an accurate intersection of edge-boundaries in future work.

5.5 Complexity Summary

Table 5.1: time complexity of all components in the project

Component	Time	Space	Guarantee	Status
Gonzalez greedy farthest-point	O(nk)	O(n)	2× optimal	implemented
Multi-start Gonzalez, m restarts	O(mnk)	O(n)	2× optimal	implemented
Hochbaum–Shmoys parametric pruning	$O(n^2 \log n)$	$O(n^2)$	2× optimal	implemented
Brute-force exact optimum	$O(C(n,k)\cdot nk)$	$O(n^2)$	exact	implemented
Local search swap polish	$O(t\cdot n^2k)$	O(n)	none (≤ start)	implemented
Random centres baseline	O(nk)	O(k)	none	implemented
k-means (Lloyd + k-means++)	O(nkI)	O(n + k)	different objective	implemented
Delaunay triangulation (Qhull)	O(n log n)	O(n)	exact	implemented
Delaunay triangulation (browser)	$O(n^2)$	O(n)	exact	implemented
Seed: densest via KD-tree	O(n log n)	O(n)	—	implemented
Seed: MST centre, dense matrix	$O(n^2)$	$O(n^2)$	—	implemented
Seed: MST centre via Delaunay	O(n log n)	O(n)	—	implemented
Outlier score from mesh (1-NN, MAD)	O(n)	O(n)	—	implemented
Robust Gonzalez, skip z farthest	O(nk)	O(n)	none	implemented
Delaunay filter + Gonzalez	O(n log n + nk)	O(n)	none	implemented
Charikar outlier greedy	$O(n^2k \log n)$	$O(n^2)$	3× optimal	implemented
Largest empty circle via circumcentres	O(k log k)	O(k)	exact interior	implemented
Farthest point by grid scan	$O(res^2\cdot k)$	$O(res^2)$	approximate	implemented
Capacitated / fair / streaming k-centre	—	—	—	not built

N = points, k = centres, m = restarts, I = k-means iterations, z = max. Number of outliers, res = grid resolution.

CHAPTER 6. PROPOSED FLOWCHART / BLOCK DIAGRAM / DFD

6.1 System Block Diagram

It consist of an input stage(a generator of five synthetic families with planted labels) a geometry stage (the Qhull triangulation that supplies the MST-centre seeder, 1 NN outlier scorer and circumcentres search) a solver stage (gonzalez, multi-start, local search polish, and the various comparison solver) and an evaluation stage (radius, time, precision/recall/F1, and fitted exponent).

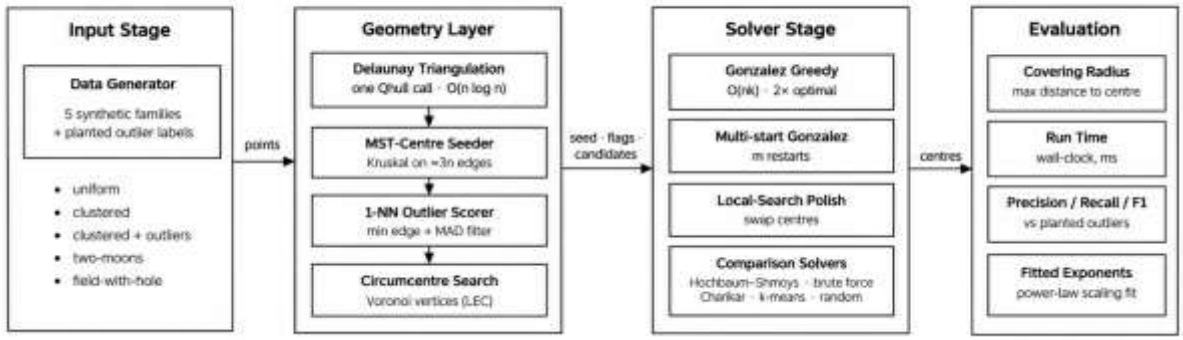


Figure 6.1:Component block diagram.

6.2 Execution Flowchart

A run generates or loads points, builds the triangulation once, chooses the first centre with the selected seeding rule, optionally removes outliers above the MAD threshold, runs Gonzalez for $k - 1$ further centres, optionally applies local-search swap polish and finally records the covering radius over all points and the run time.

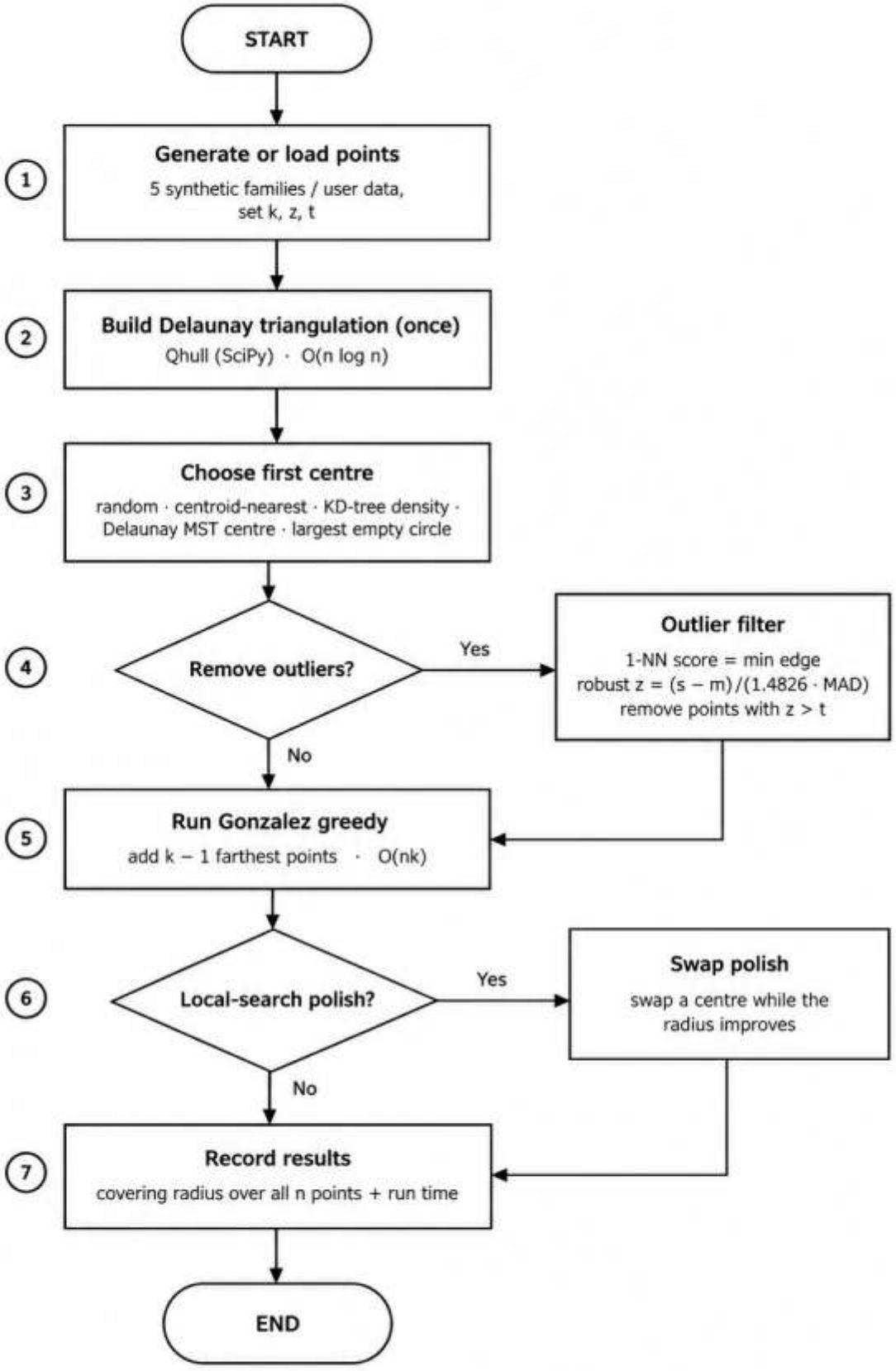


Figure 6.2: Execution flowchart of the proposed pipeline.

6.3 Data Flow Diagram – Level 0

The context diagram has two external entities, the data source (points and optional labels) and the evaluator. The single process receives points and parameters (k,z, MAD threshold) and returns centres, covering radius, outlier flags and timings.The context diagram defines two external entities, the source of the data (points and optional labels) and the on-line evaluator. There is only one process, which accepts points and parameters (k, z, MAD threshold) and outputs the centres, covering radius, outlier flags, timings.

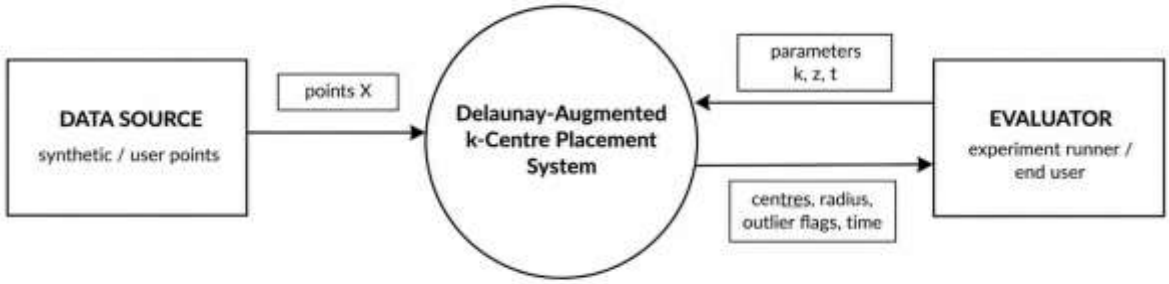


Figure 6.3: Data Flow Diagram (Level 0).

CHAPTER 7. IMPLEMENTATION (TOOLS AND TECHNOLOGY USED)

7.1 Technology Stack

The main implementation is written in Python. NumPy handles the vectorised distance calculations used in Gonzalez's update step and in the dense MST baseline.A KD-tree is used for the densest-point seed and nearest-neighbour baselines, while Matplotlib is used to create the figures. The project also includes a browser-based simulator that runs locally without a server. It allows users to place points and step through Gonzalez's algorithm; its Bowyer–Watson triangulation [7][8] takes $O(n^2)$ time because point location is handled naively.

7.2 Code Structure

The implementation is organised into a modular Python package named kcenter. Its modules isolate the main k-centre algorithms from dataset generation, geometric computations, outlier processing
Seeding techniques and visualisation. The main demonstration is carried out using rundemo.py while makepdf.py generates the project explanation document. It is possible to run the interactive simulator from the page.
View web/index.html above.Divide these tasks into separate modules so it is easier to understand, maintain and reuse the code.

The modules and the functions that they serve are shown in Table 7.1.

Table 7.1: Code structure

File / Module	Description
kcenter/core.py	Gonzalez greedy algorithm, multi-start runs and local-search refinement
kcenter/datasets.py	Creates the synthetic datasets used in the experiments
kcenter/geometry.py	Delaunay triangulation, geometric calculations and circumcentre search
kcenter/outliers.py	Outlier detection and methods for handling outliers.
kcenter/seeding.py	First-centre selection and seeding strategies
web/index.html	Interactive simulator that runs in the browser

7.3 Key Implementation Details

The same triangulation is constructed once and used for centre seeding, outlier scoring and the farthest-point search. For each vertex, the minimum incident-edge length is collected in one sweep over all edges, then normalised with median and MAD (rescaled toL 1.4826). (2) During the circumcentre search, candidate points outside the map are dismissed and points near the map boundary are sampled – this boundary sampling is responsible for the discrepancy with the grid-based result of 0.02%. The Python demonstration reproduces the cited values and figures from the project archive.

CHAPTER 8. RESULT DISCUSSION AND ANALYSIS

8.1 Which Point Should Be the First Centre?

While the initial point does not influence the 2× guarantee of Gonzalez, it can nonetheless have a significant impact on the radius that is achieved in practice as mentioned in Table 8.1 and Figure 8.1.

Table 8.1: Seeding-rule findings and evidence

Finding	Evidence
The starting point changes the radius substantially	25 random starts in one instance span a 25–40% range of final radii
No deterministic rule beats a random start on most datasets	Best deterministic rules win on only 2 of 5 families
Multi-start dominates	Best on 5 of 5 families, at 25× the work for roughly 10–20% smaller radius
Geometry wins on non-convex data	On two-moons, Delaunay MST centre is the best deterministic rule (1.08× vs 1.12× for centroid-nearest)
Delaunay MST seed is 10× cheaper	19.8 ms dense $O(n^2)$ vs 1.9 ms Delaunay $O(n \log n)$ at n = 400, same point

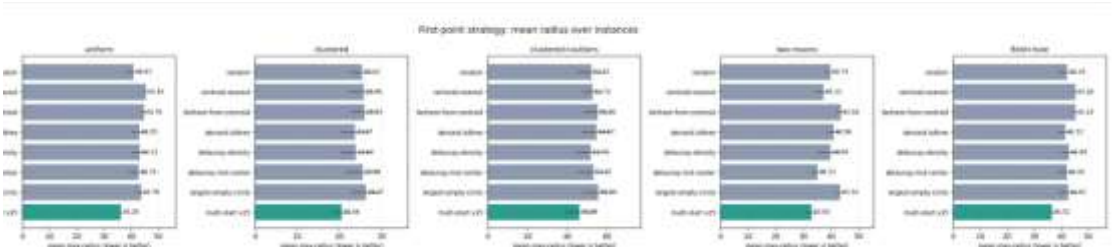


Figure 8.1: First-point strategy, mean radius over 12 instances per family (whiskers: standard deviation; lower is better).

Although the seeding results do not identify one rule that is best for every dataset, they are still informative. They are consistent with the absence of a universally best starting point and show that geometric information can help on particular data shapes.

8.2 Removing Outliers

With k =6 and 10 deliberately planted outliers, plain Gonzalez selects centres near some isolated points and produces a radius of 61.5.Applying the Delaunay filter before Gonzalez reduces the radius to 31.6 as mention in Figure 8.2.The choice of outlier score also matters: replacing the mean incident edge length with the minimum incident edge length increases F1 from 0.63 to 0.89 at MAD threshold of 8.0 as mentioned in Table 8.2.

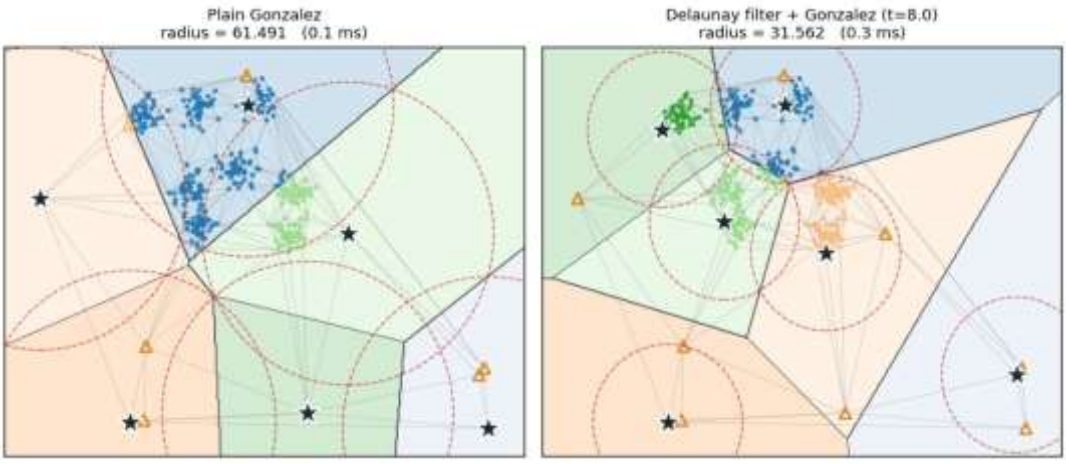


Figure 8.2: plain Gonzalez (left, radius 61.491) vs Delaunay filter + Gonzalez (right, radius 31.562, t=8.0).

Table 8.2: Outlier detector versus MAD threshold

MAD threshold	Precision (min)	Recall (min)	F1 (min)	F1 (mean)
3.5	0.42	0.96	0.58	0.29
5.0	0.67	0.93	0.77	0.35
6.0	0.76	0.91	0.83	0.38
8.0	0.91	0.89	0.89	0.45
12.0	0.95	0.84	0.88	0.57
14.0	1.00	0.79	0.87	0.63

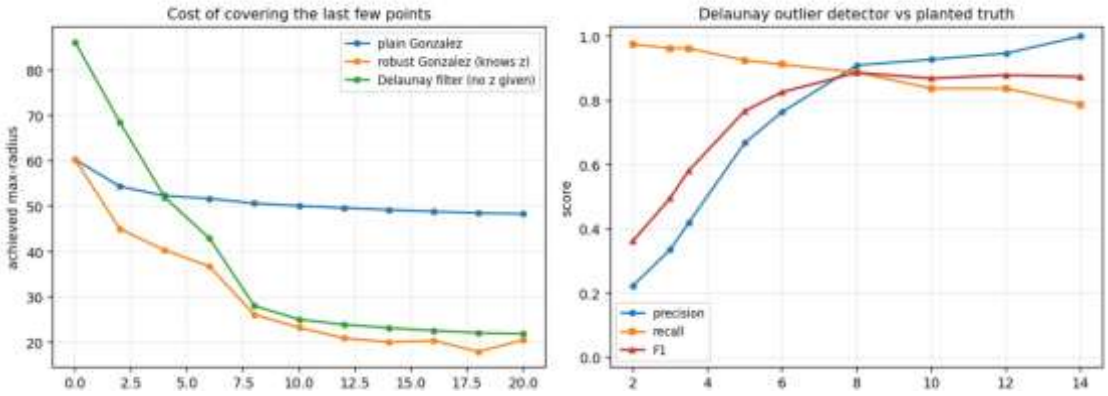


Figure 8.3: Left, Radius as more points z may remain uncovered. Right, detector precision, recall and F1 as a function of the threshold.

If 20 of the 400 points (5%) are not covered the radii reduces by around 20%. At z=0, the filter can look worse than plain Gonzalez since the point removed are still counted in the final radius calculation.From z=4 upwards the result becomes better.

Table 8.3 Comparing this tradeoff to others Charikar's method has a guaranteed smaller radius and a 3× guarantee, while the Delaunay filter does not need z to be specified

And recycles the already constructed triangulation for seed insertion and farthest-point search.

Table 8.3: Outlier-handling methods compared (n = 250, z = 8)

Method	Radius	Time	Complexity	Guarantee	Needs z?
Charikar greedy	24.55	3.2 ms	$O(n^2k \log n)$	3× optimal	yes
Robust Gonzalez	26.57	0.1 ms	$O(nk)$	none	yes
Delaunay filter + Gonzalez	30.96	1.9 ms	$O(n \log n + nk)$	none	no

8.3 Delaunay Dual as a Speed-Up for the Greedy Step

Table 8.4: Grid scan versus Delaunay circumcentres

Approach	Candidates scored	Time	Complexity	Radius found
Grid scan, 500 × 500	250,000	18.3 ms	$O(res^2 \cdot k)$	36.359
Delaunay circumcentres	404	0.3 ms	$O(k \log k)$	36.353

For these examples, the Delaunay method processes 619 times fewer candidates, and runs 55 times faster than grid scan. These two methods agree to 99.98%, with the tiny difference coming from the sampling of points along the boundary.

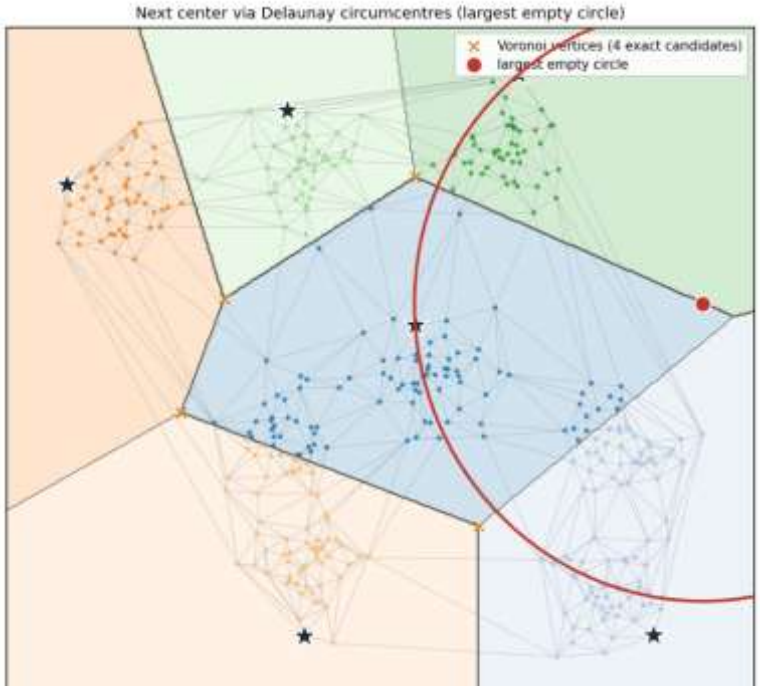


Figure 8.4: Next centre via Delaunay circumcentres (crosses: Voronoi vertices; red circle: largest empty circle).

8.4 Measured Time Complexity

All the routines were sampled six times for values of n between 100 and 3200 and the exponent fitted to the six samples for the measurements with n 400. Since the power-law fit does not explicitly include the logarithm an O(n log n) routine will have an exponent slightly above 1.

Table 8.5: Stated complexity versus fitted exponent

Routine	Stated	Fitted exponent	Verdict
Delaunay mesh	O(n log n)	1.05	matches
Delaunay MST seed	O(n log n)	1.02	matches
Dense MST seed	$O(n^2)$	2.25	matches
Hochbaum–Shmoys	$O(n^2 \log n)$	2.00	matches

The fitted exponents from the two MST implementations make the strongest comparison: 2.25 for the dense implementation and 1.02 for the Delaunay implementation. All these measurements support the expected.

In $O(n^2)$ to O(n log n). The fitted exponent of 0.78 for Gonzalez is lower than the O(nk) expectation because the Python overhead is relatively more significant in the smaller test cases.

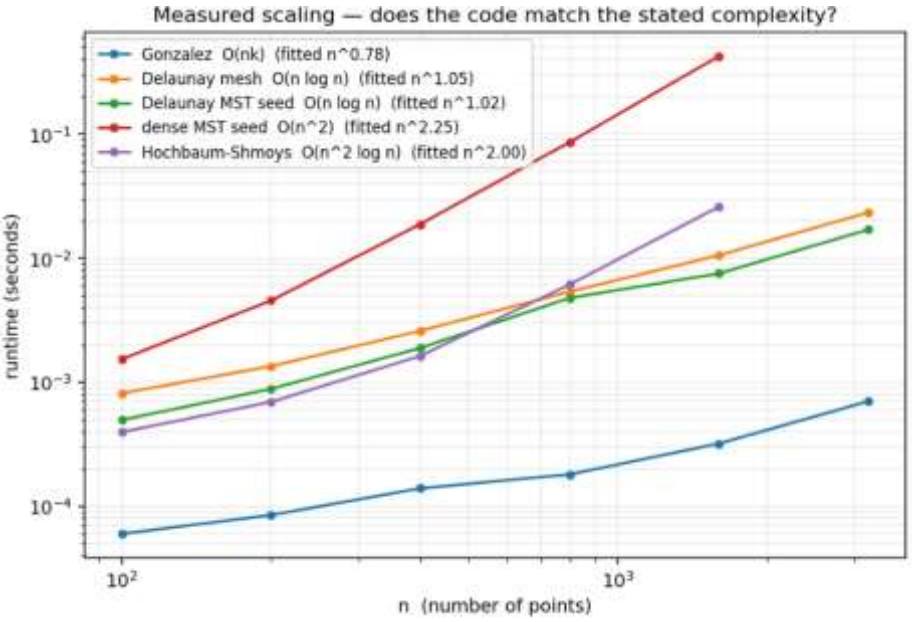


Figure 8.5. Measured runtime versus n (log-log). Since the two lines for seeding MSTs diverge as n increases,.

CHAPTER 9. CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

This project used Gonzalez's greedy farthest-point algorithm for k-centre, enhanced with a layer of extra information that used Delaunay triangulation. The extra layer does not improve the general 2× Approximation guarantee while the measured solutions were on average 1.54× the optimum, 1.89× the maximum observed and 1.05× the maximum after local-search. The actual advantage is obtained by replacing three expensive scans by a single triangulation, as shown through experiments.

9.2 Key Finding Summary

- No cheap deterministic seedings rule was able to beat a random starting point. The Delaunay MST-centre seed was roughly 10.3 times faster and produced the same point, and proved to be the most powerful deterministic rule on the two-moons data. Minimum incident-edge length as 1-NN score.

- Minimum incident-edge length as 1-NN score increased the outlier-detection F1 to 0.89 from 0.63. The filter was able to find a smaller radius than Charikar's, but it was slower and it knew z from the start. The filter beat plain Gonzalez once z=4.

- The circumcentre search ran 55 times faster than the grid scan, with 99.98% agreement between their results.
- The fitted exponents—1.02 for the Delaunay MST seed and 2.25 for the dense MST seed support the expected complexity improvement.The largest remaining bottlenecks are the $O(n^2)$ routines: the dense MST, and the Hochbaum-Shmoys and Charikar implementations.

9.3 Limitations

8.0 MAD was chosen based on the same data on which the detector was tested. A different set of data must be used as a tuning set and test set to evaluate the generalisation of the threshold.
The tests are based exclusively on simulated data. Real data could have cluster errors, repeated points or similar pattern which our tests do not address.
Twelve implementations per data set family is insufficient to show that two seeding rules are equivalent. Multi-start consumes 25 fold the computing time, so a more reasonable comparison would give both methods the same time budget.
The (browser) Delaunay implementation runs in $O(n^2)$, the Qhull implementation used in python is O(n log n).

9.4 Future Scope

It might be worthwhile to benchmark the local-search swap method (whose ratio was better at 1.05 than the simple Gonzalez at 1.54) by replacing sample boundary point with actual Voronoi edge or boundary intersection. The model could also be expand to incorporate capacity restriction at a facility, incremental or streaming placement, and road-network distances further adding realism to the classical facility location setting.It would be interesting to generate test instances on which the Gonzalez method can close in on its 2×bound (the observed Euclidean gap is 1.82)rather than note its outlier threshold on a hold-out set.We could more confidently draw conclusions from such experiments by running them on real data and comparing methods within similar time limits

REFERENCES

[1] T. F. Gonzalez, "Clustering to minimize the maximum intercluster distance," Theoretical Computer Science, vol. 38, pp. 293-306, 1985.

[2] D. S. Hochbaum and D. B. Shmoys, "A best possible heuristic for the k-centre problem," Mathematics of Operations Research, vol. 10, no. 2, pp. 180-184, 1985.

[3] M. Charikar, S. Khuller, D. M. Mount, and G. Narasimhan, "Algorithms for facility location problems with outliers," in Proc. 12th ACM-SIAM Symposium on Discrete Algorithms (SODA), 2001, pp. 642-651.

[4] B. Delaunay, "Sur la sphère vide," Izvestia Akademii Nauk SSSR, Otdelenie Matematicheskikh i Estestvennykh Nauk, no. 7, pp. 793-800, 1934.

[5] M. de Berg, O. Cheong, M. van Kreveld, and M. Overmars, Computational Geometry: Algorithms and Applications, 3rd ed. Berlin: Springer, 2008.

[6] C. B. Barber, D. P. Dobkin, and H. Huhdanpaa, "The Quickhull algorithm for convex hulls," ACM Transactions on Mathematical Software, vol. 22, no. 4, pp. 469-483, 1996.

[7] A. Bowyer, "Computing Dirichlet tessellations," The Computer Journal, vol. 24, no. 2, pp. 162-166, 1981.

[8] D. F. Watson, "Computing the n-dimensional Delaunay tessellation with application to Voronoi polytopes," The Computer Journal, vol. 24, no. 2, pp. 167–172, 1981.

[9] J. B. Kruskal, "On the shortest spanning subtree of a graph and the traveling salesman problem," Proceedings of the American Mathematical Society, vol. 7, no. 1, pp. 48-50, 1956.

[10] D. Arthur and S. Vassilvitskii, "k-means++: The advantages of careful seeding," in Proc. 18th ACM-SIAM Symposium on Discrete Algorithms (SODA), 2007, pp. 1027-1035.

[11] S. P. Lloyd, "Least squares quantization in PCM," IEEE Transactions on Information Theory, vol. 28, no. 2, pp. 129-137, 1982.

[12] P. Virtanen et al., "SciPy 1.0: Fundamental algorithms for scientific computing in Python," Nature Methods, vol. 17, pp. 261-272, 2020.

PLAG REPORT

kCenter_Report_compressed.pdf

Atharv and Jayesh

Minor Project

Indian Institute of Information Technology BP

Document Details

Submission ID

trn:oid:::1:3685196135

Submission Date

Oct 20, 2026, 6:24 PM GMT+5:30

Download Date

Oct 20, 2026, 6:26 PM GMT+5:30

File Name

kCenter_Report_compressed.pdf

File Size

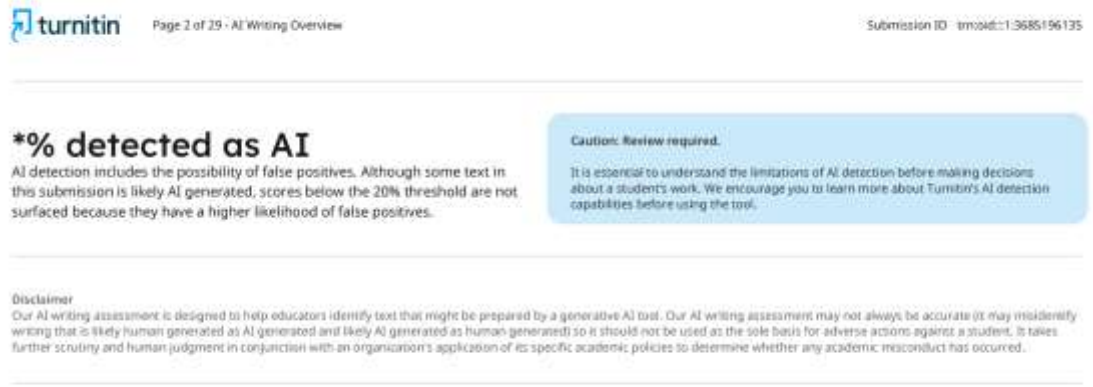
527.5 KB

27 Pages

5,406 Words

27,669 Characters

AI Plagiarism Report



turnitin Page 2 of 29 - AI Writing Overview Submission ID trn:oid:::1:3685196135

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

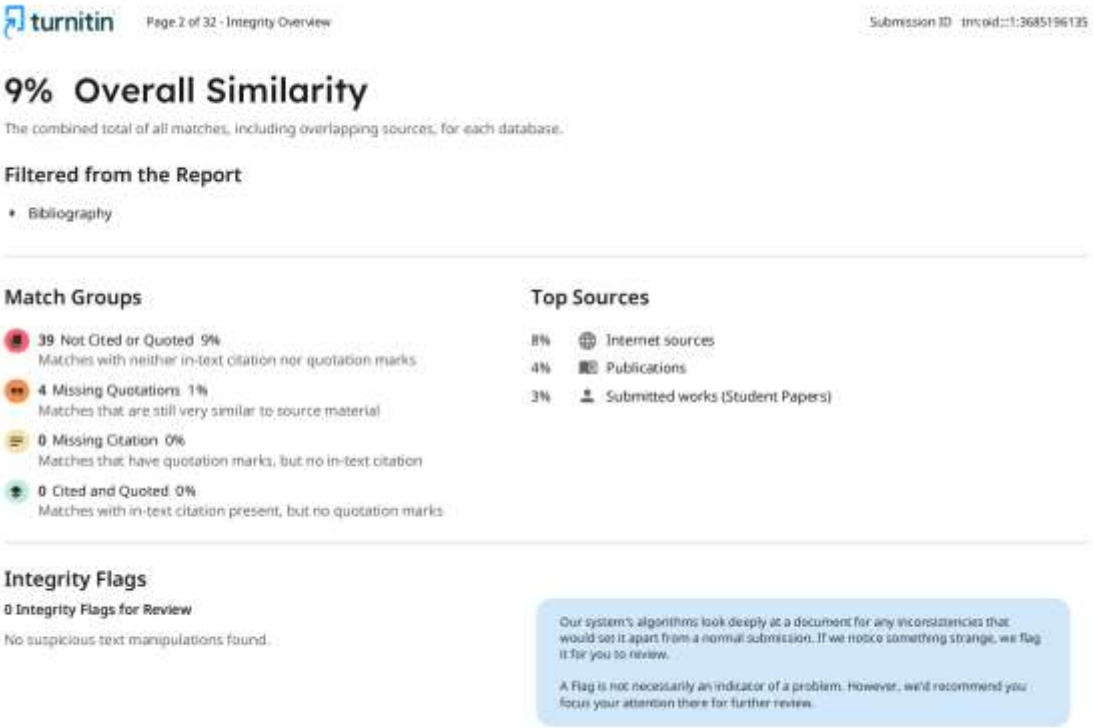
Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely human generated as AI generated and likely AI generated as human generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Similarity Plagiarism Report



turnitin Page 2 of 32 - Integrity Overview Submission ID trn:oid:::1:3685196135

9% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

- 39 Not Cited or Quoted 9%
  Matches with neither in-text citation nor quotation marks
- 4 Missing Quotations 1%
  Matches that are still very similar to source material
- 0 Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

8% Internet sources
4% Publications
3% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.